Data Structures — Core Exam Points: Final Review Handout, Volume 2 (Modern C++ Edition)

> **Revision note**: this document is a fully de-noised reconstruction of the original exam-point compilation. It corrects the many logical gaps and spelling errors of the original draft; all pseudocode has been stripped of raw `malloc / free` and C-style macro definitions and standardized to modern C++11.

Introduction: Pitfall-Avoidance Guide for the Final Exam (annotated by Akiro)

1. **The written-exam confusion between "head pointer" and "head node"**: multiple-choice questions on finals love testing the **emptiness check of a linked list**. The first second you get the paper, check whether the question asks about a singly linked list **without** a head node (emptiness condition `head == nullptr`) or a singly linked list **with** a head node (emptiness condition `head->next == nullptr`)!
2. **The essential difference between an algorithm and a program**: a program may loop forever without end (for example, an operating system's event-listening kernel), but **an algorithm must have finiteness** (it must halt within a finite number of steps and a finite time). A guaranteed fill-in-the-blank point.

Chapter 1 Overview of Data Structures

1.1 Basic Concepts and the Terminology Hierarchy

- **Data**: a symbolic representation of objective things; in computer science, the collective term for all symbols that can be input into a computer and processed by computer programs.
- **Data element**: the **basic unit** of data, existing as an individual within a collection; also called an element, node, vertex, or record.

  *(Supplement: a data element can be composed of several **data items**. A data item is the smallest indivisible unit of data.)*
- **Data object**: a collection of data elements with the **same properties**; a subset of data (in object-orientation it often corresponds to the notion of "attributes").
- **Data structure**: a collection of data elements that have one or more specific relationships among one another.

  *(Mathematical abstraction: the **two-tuple** is a finite set used to represent a data object together with the relationships among its elements.)*

1.2 Mapping from Logical Structure to Storage Structure

Plaintext

```
Logical structure of data (independent of computer memory)
 ├─ Set structure : apart from "belonging to the same set", no other relations among elements
 ├─ Linear structure : one-to-one relations among elements
```

```
  ├─ Tree structure : one-to-many relations among elements (root excepted: one predecessor, many
successors)
  └─ Graph structure : many-to-many relations among elements
```

For a computer to process data, the logical structure must be mapped onto the following two **physical storage structures**:

1. **Sequential storage structure**: store logically adjacent elements in physically adjacent memory cells, expressing logical relations through relative positions.
2. **Linked storage structure**: express logical relations through pointers; physical adjacency is not required.

1.3 Time/Space Complexity of Algorithms and Evaluation Criteria

- **Comparison of common time-complexity classes (a guaranteed ordering question on the paper)**:

$$O(1) < O(\log_2 n) < O(n) < O(n\log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$$

- **The five properties of an algorithm**: ① input, ② output, ③ **finiteness**, ④ **definiteness** (no ambiguity), ⑤ **feasibility**.
- **The five requirements of algorithm design**: ① correctness, ② **robustness** (responds properly to illegal data), ③ readability, ④ modifiability and extensibility, ⑤ high efficiency.

Chapter 2 Linear Lists (Refactoring in Practice)

2.1 Core Theory Quick Notes

- **Endpoint boundaries**: in a linear list, the first element has no immediate predecessor and the last element has no immediate successor.
- **Pointer deletion**: in a singly linked list, if `p` is the immediate predecessor of `q`, the standard deletion of `q` is: `p->next = q->next; delete q;`.
- **Cost accounting**: in a sequential list of length N, inserting a new element moves on average $\frac{N}{2}$ elements; deleting an element moves on average $\frac{N-1}{2}$ elements.
- **Choice by scenario**: if the main operations on the linear list are insertions and deletions at the end, **sequential list** storage is the most time-saving.
- **Solving for addresses**: suppose each element of a sequential list occupies 3 cells and the address of the 13th element (index 12) is 336; then the base address is $LOC = 336 - 12 \times 3 = 300$.

2.2 Object-Oriented C++ Refactor of the Singly Linked List with a Head Node

> The original draft contained the `previos` misspelling and mixed raw-pointer usage; the standard engineering form is as follows:

C++

```cpp
#include <iostream>

typedef int DataType;

struct ListNode {
    DataType data;
    ListNode* next;
    ListNode(DataType val = 0) : data(val), next(nullptr) {}
};

class SinglyLinkedList {
private:
    ListNode* head;

    // internal helper: find the k-th node
    ListNode* getElem(int k) {
        if (k < 0) return nullptr;
        ListNode* p = head;
        for (int i = 0; p != nullptr && i < k; ++i) {
            p = p->next;
        }
        return p;
    }

public:
    SinglyLinkedList() {
        head = new ListNode(); // initialize a singly linked list with a head node
    }

    // insert element x before position i (1-based logical convention)
    bool insert(int i, DataType x) {
        ListNode* q = getElem(i - 1); // find node i-1
        if (q == nullptr) {
            std::cout << "Invalid insertion position!\n"; return false;
        }
        ListNode* s = new ListNode(x);
        s->next = q->next;
        q->next = s;
        return true;
    }

    // delete the element at position i
    bool remove(int i) {
        ListNode* q = getElem(i - 1);
        if (q == nullptr || q->next == nullptr) {
            std::cout << "Invalid deletion position!\n"; return false;
        }
        ListNode* p = q->next;
        q->next = p->next;
        delete p; // C++ memory deallocation
        return true;
    }

    ~SinglyLinkedList() {
        ListNode* p = head;
        while (p != nullptr) {
```

```
            ListNode* tmp = p->next;
            delete p;
            p = tmp;
        }
    }
};
```

2.3 A Grand Compilation of High-Frequency Multiple-Choice Exam Points (Compiled by Akiro)

Exam-point description

Core of what the question asks

Correct standard conclusion

Source tracing

Linked-list emptiness check

Emptiness condition of the singly linked list **without** a head node

`head == nullptr` (the original question's answer was A)

Linked-list emptiness check

Emptiness condition of the singly linked list **with** a head node

`head->next == nullptr` (answer B)

Atomic post-insertion logic

Insert node `s` **after** node `p`

`s->next = p->next; p->next = s;` (answer B)

Atomic deletion logic

Delete the node **after** node `p`

`p->next = p->next->next;` (answer A)

Search in an ordered list

Search for a given element in an ordered singly linked list of n nodes

on average $\frac{n+1}{2}$ comparisons (about $\frac{n}{2}$; answer B)

Time complexity

Building an ordered singly linked list from n vectors

$O(n^2)$ *(note: the original document's answer printed as B is wrong; rigorously it should be* $O(n^2)$*)*

Time complexity

Inserting after a known node / inserting after a given value

insertion after a known node is $O(1)$; insertion after a searched value is $O(n)$

Long-Answer Question: Comparing the Pros and Cons of Sequential Lists and Linked Lists

- **Sequential list**: fast access at $O(1)$, but once its space is defined it is hard to expand; moreover, insertions and deletions require moving nearly half the elements on average, so modification efficiency is not high.
- **Linked list**: dynamic memory allocation, **high modification efficiency** (only pointers need to be modified; the physical data never moves). But it can only be accessed sequentially along the chain, so **access efficiency is low**, $O(n)$.
- **Selection conclusion**: if the total size of the list is basically stable with few insertions and deletions, but **ultra-fast access** is required, always choose the **sequential list**.

Chapter 3 Stacks and Queues

3.1 Core Properties and the Big Decision Table

- **Essential characteristics**: a stack is **first-in, last-out (LIFO)**; a queue is **first-in, first-out (FIFO)**.
- **Operational efficiency**: whether stored sequentially or linked, insert/delete operations on stacks and queues always have time complexity $O(1)$.
- **Dequeue vs. get-front**: dequeue completely **deletes** the front data; getting the front only **reads the value** without deleting.
- **Core advantage of the linked stack**: compared with the sequential stack, a linked stack **never becomes full** (unless system memory is exhausted; the multiple-choice answer is D).
- **Impossible pop-sequence reasoning**: with the push sequence `ABCDE`, the pop sequence that absolutely cannot occur is `EDACB` (the multiple-choice answer is B).

(Pitfall verification: if E pops first, then A, B, C, D, E have all been pushed; from then on the pop order is locked to EDCBA — it is impossible for D to pop and then skip C to pop A directly.)

3.2 Standard C++ Implementations of the Linked Queue and Linked Stack

Exam Point 1: Linked-Queue Encapsulation (LinkQueue)

C++

```
struct QNode {
    int data;
    QNode* next;
    QNode(int val = 0) : data(val), next(nullptr) {}
};

class LinkedQueue {
private:
    QNode* front;
    QNode* rear;
public:
    LinkedQueue() {
        front = rear = new QNode(); // attach a head node
    }
```

```
    void enQueue(int e) {
        QNode* p = new QNode(e);
        rear->next = p;
        rear = p;
    }

    bool deQueue(int& e) {
        if (front == rear) return false; // queue empty
        QNode* p = front->next;
        e = p->data;
        front->next = p->next;

        if (rear == p) rear = front; // if the deleted node was the rear, point the rear pointer
back to the head node
        delete p;
        return true;
    }
};
```

Exam Point 2: Linked-Stack Encapsulation (LinkedStack)

C++

```
struct StackNode {
    int data;
    StackNode* next;
    StackNode(int val = 0, StackNode* p = nullptr) : data(val), next(p) {}
};

class LinkedStack {
private:
    StackNode* top; // a linked stack needs no head node; top points directly to the valid stack
top
public:
    LinkedStack() : top(nullptr) {}

    bool isEmpty() const {
        return top == nullptr;
    }

    void push(int val) {
        top = new StackNode(val, top);
    }

    bool pop(int& val) {
        if (isEmpty()) return false;
        StackNode* p = top;
        val = p->data;
        top = top->next;
        delete p;
        return true;
    }
};
```

Chapters 4 & 5: Strings, Arrays, and Generalized Lists

- **Definition of a string**: a finite sequence of zero or more characters. Assignment form: `string x = "abc";`.
- **Multi-dimensional array exam point**: on the paper, be sure to sort out the address-computation conversions between **row-major** and **column-major** storage.
- **Sparse-matrix storage options**: ① triplet table; ② two-tuple with an auxiliary row vector; ③ **orthogonal (cross) linked list**.

Refactored Orthogonal (Cross) Linked-List Matrix Structure

C++

```
struct OLNode {
    int row, col; // row and column indices
    int value;    // element value
    OLNode *right, *down; // row successor and column successor pointers
};

struct CrossList {
    int mu, nu, tu; // matrix row count, column count, nonzero element count
    OLNode** rhead; // base address of the row header pointer array
    OLNode** chead; // base address of the column header pointer array
};
```

Chapter 6 Trees and Binary Trees

6.1 Distinguishing Basic Concepts of Trees and Binary Trees

- **Degree**: a node's degree = the number of subtrees it owns; the tree's degree = the **maximum** of the degrees of all nodes in the tree.
- **Lineage along paths**: ancestors and descendants are delimited by connected paths. Data structures by default deal with **ordered trees**.
- **Critical knockout point**: a general tree has only preorder, postorder, and level-order traversals; **a general tree absolutely has no inorder traversal** (because its subtrees have no left/right distinction)!

6.2 Core Property Formulas of Binary Trees

1. **Full binary tree**: leaves appear only on the bottom level, and the tree contains **only nodes of degree 0 and degree 2**.
2. **Complete binary tree**: leaves are confined to the **bottom two levels and to the left**. There is **at most one node of degree 1, and it must be a left child**.
3. **Key property theorems**:
    - Level i holds at most 2^{i-1} nodes.
    - A binary tree of depth k has at most $2^k - 1$ nodes.

- **Leaf formula**: $n_0 = n_2 + 1$.
- Depth of a complete binary tree with n nodes: $\lfloor \log_2 n \rfloor + 1$.

6.3 Classic Binary-Tree Problem Drills (Detailed by Akiro)

Example 1: Counting the Leaves of a Complete Binary Tree

> **Problem**: a complete binary tree is known to have 47 nodes; find its number of leaf nodes.

Standard solution walkthrough:

1. Depth derivation: five full levels hold $2^5 - 1 = 31$ nodes in total, six full levels hold 63. Hence the tree has depth 6.
2. Levels 1 through 5, when full, hold 31 nodes in total.
3. The number of leaves on level 6 (the bottom level) is: $47 - 31 = 16$.
4. Working backward from the 16 children on level 6, they occupy $\frac{16}{2} = 8$ parent nodes on level 5.
5. Level 5 should have $2^{5-1} = 16$ nodes in total; 8 are occupied, leaving $16 - 8 = 8$ leaves.
6. Total leaves = level-6 leaves + level-5 leaves = $16 + 8 = \mathbf{24}$ **in total** (the original question's answer: C).

Example 2: Uniquely Reconstructing the Tree Structure

> **Problem**: a binary tree has preorder `ABCDEFG` and inorder `CBEDAFG`; draw the tree.

Plaintext

```
Reconstructed structure diagram (the figure for the original question's answer C):
       A
      / \
     B   F
    /     \
   C       G
    \     /
     E   D
```

Example 3: Extreme Binary-Tree Shapes

- Preorder == inorder ⇒ the tree has **only right subtrees**;
- Inorder == postorder ⇒ the tree has **only left subtrees**;
- Preorder == postorder ⇒ the tree has **only a single root node** or is empty.

The Recursive Low-Level Form of Preorder Traversal of a Binary Tree

C++

```
struct TreeNode {
    char data;
```

```
    TreeNode *lchild, *rchild;
};

void preOrder(TreeNode* root) {
    if (root != nullptr) {
        std::cout << root->data; // visit the root first
        preOrder(root->lchild);  // recurse into the left subtree
        preOrder(root->rchild);  // recurse into the right subtree
    }
}
```

Chapters 7–10: A Quick Tour of Graphs and Search Tables

7.1 Basic Graph Concepts

- **Complete-graph edge-count formula**: an undirected complete graph on n vertices has $\frac{1}{2}n(n-1)$ edges; a directed complete graph has $n(n-1)$ arcs.
- **Paths and connectivity**: a path whose start and end coincide is called a **cycle (loop)**. If a path exists between any two vertices of a graph, the graph is **connected**.
- **Logical expression of the adjacency matrix**:

$$A[i][j] = \begin{cases} w_{i,j} & \langle v_i, v_j \rangle \in VR \\ 0 & \text{if the edge does not exist} \end{cases}$$

- **Core traversals**: ① **depth-first traversal (DFS)** (modeled on tree preorder; a flag marks visited vertices); ② **breadth-first traversal (BFS)** (uses a queue to advance level by level).

7.2 Searching and Sorting

- **Static search table**: only query and retrieval operations; the table structure is fixed at run time.
- **Dynamic search table**: nodes are dynamically inserted and deleted during the search process.
- **The essence of sorting**: rearranging an arbitrary sequence into one that is **ordered by key**.